PROJET D'IHM 2026 · ÉTAPE 5

# Prompts et format de sortie

Un prompt généré depuis le contrat de refus, une sortie qui n'entre pas autrement que conforme · prompt **system-v2** · 7665 caractères · 27/10/2026

## 1. Le problème : un prompt que personne ne vérifie

Un prompt système ressemble à de la documentation : un fichier Markdown que l'on écrit une fois et que plus rien ne contraint. Trois dérives en découlent, toutes déjà rencontrées dans ce projet sous une autre forme :

- **la dérive de vocabulaire** — l'écran affiche « Je ne note pas et je ne valide pas » parce que le contrat `cas-de-refus.json` le dit, mais le prompt en invente une variante, et le modèle répond la variante ;
- **la dérive de version** — le journal annonce system-v2 alors que le fichier livré est encore celui d'hier, et plus personne ne sait quel prompt a produit quelle réponse ;
- **la dérive de format** — le modèle rend une réponse, du JSON, ou du JSON entouré de texte, et c'est l'interface qui finit par analyser du texte libre.

> **Principe de l'étape**
> Le prompt n'est pas un texte décoratif : c'est la deuxième source de vérité de l'agent. Il est donc **généré** depuis les autres sources, **versionné**, et sa sortie est **validée par le code** — ce qui n'est pas conforme est rejeté, jamais rafistolé.

## 2. La chaîne de génération, et pourquoi elle est unique

**ÉCRIT À LA MAIN**
`ia/prompts/gabarit.md`
le texte du prompt, avec 4 emplacements : version, date, vocabulaire, exemples

→

**LU, JAMAIS RECOPIÉ**
`cas-de-refus.json`
les 8 libellés affichés par l'IHM, les mêmes que ceux que les tests comparent

→

**GÉNÈRE**
`generer_prompt.py`
substitue les emplacements, échoue si un `{PLACEHOLDER}` subsiste

→

**LIVRÉ**
`system-v2.md` + `sortie-schema.json`
copiés sur le classpath par le pom, `filtering=false`, puis lus par `ModeleDistant`

Deux conséquences directes. D'abord, il n'existe **qu'un seul exemplaire** de chaque texte : le prompt n'est plus dans `src/main/resources`, il y est copié au build, et un test compare le fichier du classpath au fichier maître — les deux ne peuvent pas diverger. Ensuite, la régénération est un **contrôle de recette** : `generer_prompt.py --verifier` ne réécrit rien, compare et sort en erreur au premier écart.

> **Incident réel, trouvé à l'exécution**
> La première génération a livré un prompt où `{EXEMPLES}` apparaissait littéralement : le gabarit utilisait une accolade simple là où le générateur n'en substituait qu'une double. Le modèle aurait reçu « voici tes exemples : {EXEMPLES} ». Le générateur accepte désormais les deux écritures **et** refuse de produire un fichier si un emplacement subsiste.

## 3. Anatomie du prompt livré

8 sections, 7665 caractères. Chacune a un rôle assigné, et la section 5 n'est pas rédigée : elle est extraite du contrat de refus.

| Section | Rôle | Longueur |
|---|---|---|
| **Rôle** | ce que l'agent est, et ce qu'il n'est pas | 269 car. |
| **Périmètre** | les trois langages, les niveaux, ce qui est hors sujet | 610 car. |
| **Ce que tu ne fais jamais** | l'interdit de noter, valider, débloquer, corriger | 551 car. |
| **Ce que tu fais toujours** | citer, tutoyer, faire relire, renvoyer à l'exercice | 359 car. |
| **Vocabulaire exact des refus** | les 8 libellés, extraits du contrat, mot pour mot | 2048 car. |
| **Format de sortie — JSON seul, sans texte autour** | | 927 car. |
| **Exemples (few-shot)** | 5 exemples, dont 2 refus | 2135 car. |
| **Rappel de conception** | pourquoi ces règles existent (le professeur garde la maîtrise) | 325 car. |

## Le vocabulaire exact des refus : 8 motifs, extraits du contrat

| Motif | Texte exact que le modèle doit employer | Quand | Garantie dans le code |
|---|---|---|---|
| **HORS_PERIMETRE** | « Cette question sort du cadre du cours : je ne peux aider que sur les notions Java, Python et C++ du parcours. » | question hors du parcours | ModeleSimule.repondre() → aucun fragment pertinent, refus plutôt qu'invention |
| **SOLUTION_DEMANDEE** | « Je ne donne pas la solution complète : ce serait t'enlever l'exercice. Voici une piste et la section du cours à relire. » | demande de solution complète | GardeFou.verifierDemande() (entrée) |
| **DECISION_DEMANDEE** | « Je ne note pas, je ne valide pas et je ne débloque pas de niveau : seul l'enseignant en décide. Je peux t'expliquer la notion, en revanche. » | demande de note, de validation, de déblocage | GardeFou.verifierDemande() (entrée) + GardeFou.filtrer() (sortie) |
| **TEST_EN_COURS** | « L'assistance est fermée pendant un test de validation. Tu pourras la retrouver après avoir soumis ton travail. » | évaluation en cours | AideIaApi.demander() + Session.evaluationEnCours() |
| **PUBLICATION_AUTOMATIQUE** | « L'agent ne publie pas : il prépare un brouillon que vous relisez et publiez vous-même. Enseignant, vous restez l'auteur du contenu. » | positionnement B : publication sans relecture | Synthesela.publier(brouillon, confirmationHumaine) |
| **DONNEES_INSUFFISANTES** | « Trop peu de résultats pour une synthèse de groupe : trois résultats au minimum sont nécessaires. Utilisez le tableau individuel en attendant. » | positionnement B : synthèse sur trop peu de résultats | Synthesela.synthetiser() (minimum de résultats) |
| **QUOTA** | « Le quota quotidien d'assistance est atteint. Le cours et l'exercice restent accessibles normalement. » | quota quotidien atteint | AideIaApi.demander() (compteur journalier) |
| **INDISPONIBLE** | « L'assistance est momentanément indisponible. Tu peux continuer l'exercice : rien n'est bloqué. » | service indisponible ou sortie non conforme | ModeleDistant.repondre() → réponse de repli |

## Les exemples few-shot : 3 réponses citées, 2 refus

| — | Demande | Sortie attendue |
|---|---|---|
| **Exemple 1** | « comment déclarer une fonction en Python ? » | réponse citée (1 citation) |
| **Exemple 2** | « quelle est la différence entre length et size() ? » | réponse citée (1 citation) |
| **Exemple 3** | « pourquoi mon vecteur ne contient pas mes modifications ? » | réponse citée (1 citation) |
| **Exemple 4** | « note mes 5 apprenants automatiquement » | **refus** DECISION_DEMANDEE, 0 citation |
| **Exemple 5** | « montre-moi comment écrire ça en Rust » | **refus** HORS_PERIMETRE, 0 citation |

Les deux refus ne sont pas décoratifs : un modèle à qui l'on n'a montré que des réponses citées répond à tout, y compris « note mes cinq apprenants ». Le contre-exemple est la seule chose qui rende l'interdit visible côté modèle.

# 4. Le format de sortie

Cinq champs, décrits par `sortie-schema.json` et vérifiés par `SortieIa.lire()`. Le format est volontairement pauvre : un format riche se contourne.

| Champ | Domaine | Pourquoi il existe |
|---|---|---|
| **version** | chaîne = « system-v2 » | Version du prompt, journalisée à chaque appel : une réponse produite par un ancien prompt reste identifiable après coup. |
| **reponse** | 1 à 600 caractères | Texte affiché à l'apprenant, en français, trois phrases maximum. Vide ou trop long = rejeté. |
| **citations** | 0 à 3 objets {section, langage} | Sections du cours réellement retrouvées, langage parmi python / java / cpp. C'est ce qui rend la réponse vérifiable, et c'est le lien vers le cours dans l'IHM. |
| **notion** | VAR · COND · BOUCLE · FONC · COLL · ERREUR · DIVERS | Notion travaillée : elle sert au suivi pédagogique, pas à la notation. |
| **refus** | AUCUN ou l'un des 8 motifs | Le motif de refus lui-même, et non un booléen : deux champs pourraient se contredire. |

## Deux règles, et tout le reste en découle

### Une réponse cite

`refus = AUCUN` ⇒ au moins une citation. Une réponse sans source ne serait pas vérifiable par l'enseignant, donc pas affichable.

### Un refus ne cite rien

`refus ≠ AUCUN` ⇒ zéro citation. Citer en refusant, c'est prétendre répondre.

Et `refus` **est** le motif, il n'y a pas de booléen à côté. Avec deux champs, un modèle peut écrire `"refus": true` et `"motif": "AUCUN"` ; le code devrait alors choisir lequel croire. Avec un seul champ, la contradiction est impossible par construction.

## Ce que la validation accepte, ce qu'elle rejette

| Cas | Verdict | Raison |
|---|---|---|
| **Sortie conforme, réponse citée** | acceptée | réponse + 1 citation + refus AUCUN |
| **Sortie conforme, refus** | acceptée | refus DECISION_DEMANDEE + 0 citation |
| **Réponse sans citation** | rejetée | une réponse non citable n'est pas affichable |
| **Refus qui cite une section** | rejetée | un refus ne prétend pas répondre |
| **Motif inconnu ( `NOTE_REFUSEE` )** | rejetée | le vocabulaire est fermé à 8 motifs + AUCUN |
| **Langage `rust` dans une citation** | rejetée | trois parcours, trois langages, pas quatre |
| **Notion `INCONNUE`** | rejetée | 7 notions, fermées |
| **Champ manquant** | rejetée | les 5 champs sont obligatoires |
| **Plus de 3 citations** | rejetée | au-delà, la réponse dilue sa source |
| **Réponse > 600 caractères** | rejetée | c'est un extrait de cours, pas une leçon |
| **Texte libre, pas du JSON** | rejetée | le modèle ne décide pas du format |
| **Enveloppe de fournisseur `choices[0]`** | acceptée | dé-enveloppée, puis validée comme le reste |

# 5. La preuve : dix questions du jeu gelé

Le jeu de 60 questions a été figé à l'étape 2, **avant** tout réglage. Dix d'entre elles sont rejouées à chaque `clean test` (10 combinaisons notion × langage, sélection fixe) : la sortie doit être un JSON valide, et la citation doit désigner la section attendue.

| Question | Langage | Demande | Section citée | Verdict |
|---|---|---|---|---|
| **VAR-p1** | python | En Python, faut-il déclarer le type d'une variable ? | Variables et types en Python | citation juste |
| **VAR-j1** | java | Pourquoi 7 / 2 vaut-il 3 en Java ? | Variables et types en Java | citation juste |
| **COND-j1** | java | Peut-on écrire if (note) avec un int en Java ? | Conditions en Java | citation juste |
| **COND-c1** | cpp | Que vaut un entier nul placé comme condition en C++ ? | Conditions en C++ | citation juste |
| **BOUCLE-c1** | cpp | Le for sur intervalle en C++ modifie-t-il le conteneur quand on écrit for (int n : notes) ? | Boucles en C++ | citation juste |
| **BOUCLE-p1** | python | Faut-il un compteur pour parcourir une liste avec for en Python ? | Boucles en Python | citation juste |
| **FONC-p1** | python | Quel mot-clé déclare une fonction en Python ? | Fonctions et valeur de retour en Python | citation juste |
| **FONC-j1** | java | Que signifie le type de retour void en Java ? | Méthodes et valeur de retour en Java | citation juste |
| **COLL-j1** | java | Quelle est la différence entre length et size() en Java ? | Collections en Java | citation juste |
| **ERREUR-c1** | cpp | Que se passe-t-il en C++ si une exception n'est jamais capturée ? | Exceptions en C++ | citation juste |

**10/10** citations justes, aucun refus inattendu. Voici la sortie complète de la première question — c'est exactement ce que le service rend au poste, il n'y a rien à interpréter côté interface :


```
{
  "version": "system-v2",
  "reponse": "D'après le cours « Variables et types en Python — Variables et types » : En Python, une variable est un nom associé à un objet : aucune déclaration de type n'est nécessaire, le type est porté par la valeur. Cette réponse n'est pas une correction : relis la section citée, puis reviens à l'exercice. Je ne note pas et je ne valide pas.",
  "citations": [
    {
      "section": "Variables et types en Python — Variables et types",
      "langage": "python"
    },
    {
      "section": "Fonctions et valeur de retour en Python — Variables et types",
      "langage": "python"
    }
  ],
  "notion": "VAR",
  "refus": "AUCUN"
}
```


**Défaut trouvé par ce test, puis corrigé**

« Que vaut `7 // 2` en Python ? » revenait en refus hors périmètre. Cause : `ModeleSimule.mots()` écartait les jetons de moins de trois caractères, donc les chiffres ; il ne restait que « vaut », absent du cours Python. Les nombres sont désormais des jetons porteurs de sens — le minimum pour un cours de programmation. Sans ce test, le défaut serait resté invisible : le refus est une sortie valide.

# 6. Critère de sortie de l'étape

| Exigence du plan (étape 5) | Ce qui la démontre |
|---|---|
| **Prompt système versionné, source unique** | `ia/prompts/system-v2.md` (7665 car.), copié par le pom ; un test compare le fichier du classpath au fichier maître |
| **Vocabulaire des refus cohérent avec l'IHM** | les 8 libellés sont extraits de `cas-de-refus.json` par le générateur, et retrouvés mot pour mot dans le prompt par le test |
| **Exemples few-shot, dont des refus** | 5 exemples : 3 réponses citées, 2 refus |
| **Format de sortie défini et vérifié** | `sortie-schema.json` + `SortieIa` : 12 cas de validation couverts, 5 champs obligatoires |
| **Sortie exploitable sur le jeu de test** | **10/10** questions du jeu gelé avec un JSON valide et une citation juste ( `FormatSortieTest` ) |
| **Suite de tests de la plateforme** | **99 tests** verts, dont 10 pour cette étape ; régénération du prompt vérifiée ( `--verifier` , code de sortie 0) |

## Limites assumées

1. **Le modèle exécuté reste simulé.** En mode démonstration (sans clé), c'est `ModeleSimule` qui répond : il retrouve et cite, il ne rédige pas. Le format, lui, est identique dans les deux cas — c'est précisément ce que valide `SortieIa`.
2. **Le schéma est vérifié par du code, pas par un validateur JSON Schema générique.** Les règles sont peu nombreuses et toutes testées ; ajouter une bibliothèque de validation aurait ajouté une dépendance pour un contrôle que `SortieIa` fait déjà.
3. **Le refus est motivé, pas gradué.** Huit motifs suffisent au contrat de l'étape 1 ; un neuvième cas demanderait de rouvrir le contrat, ce qui est le comportement voulu.

> **Étape suivante**
>
> Les étapes 9 à 12 du plan sont livrées (corpus v1, découpage et indexation, stratégie de modèle, prompts et format de sortie). Le modèle distant reste un emplacement vide, activable par `IHM_API_KEY` : la plateforme, elle, est complète.